

FGV EMap
João Pedro Jerônimo

Aprendizado Profundo

Revisão para A1

Rio de Janeiro
2026

Conteúdo

1 Segmentação Semântica	3
1.1 Introdução	4
1.2 Métricas	4
1.3 Abordagens progressivas	5
1.4 Upsampling	7
1.4.1 Max Unpooling	7
1.4.2 Transpose Convolution	8
1.5 Contexto Global e Atrous Convolution	9
1.6 Blocos Residuais	9
1.7 Arquiteturas	10
1.7.1 SegNet	10
1.7.2 U-Net	10
1.7.3 ResUNet	11

Segmentação Semântica

1.1 Introdução

A segmentação semântica é uma tarefa de visão computacional que consiste em classificar cada pixel de uma imagem em uma categoria específica. Essa tarefa é fundamental para diversas aplicações, como direção autônoma, análise médica e realidade aumentada.

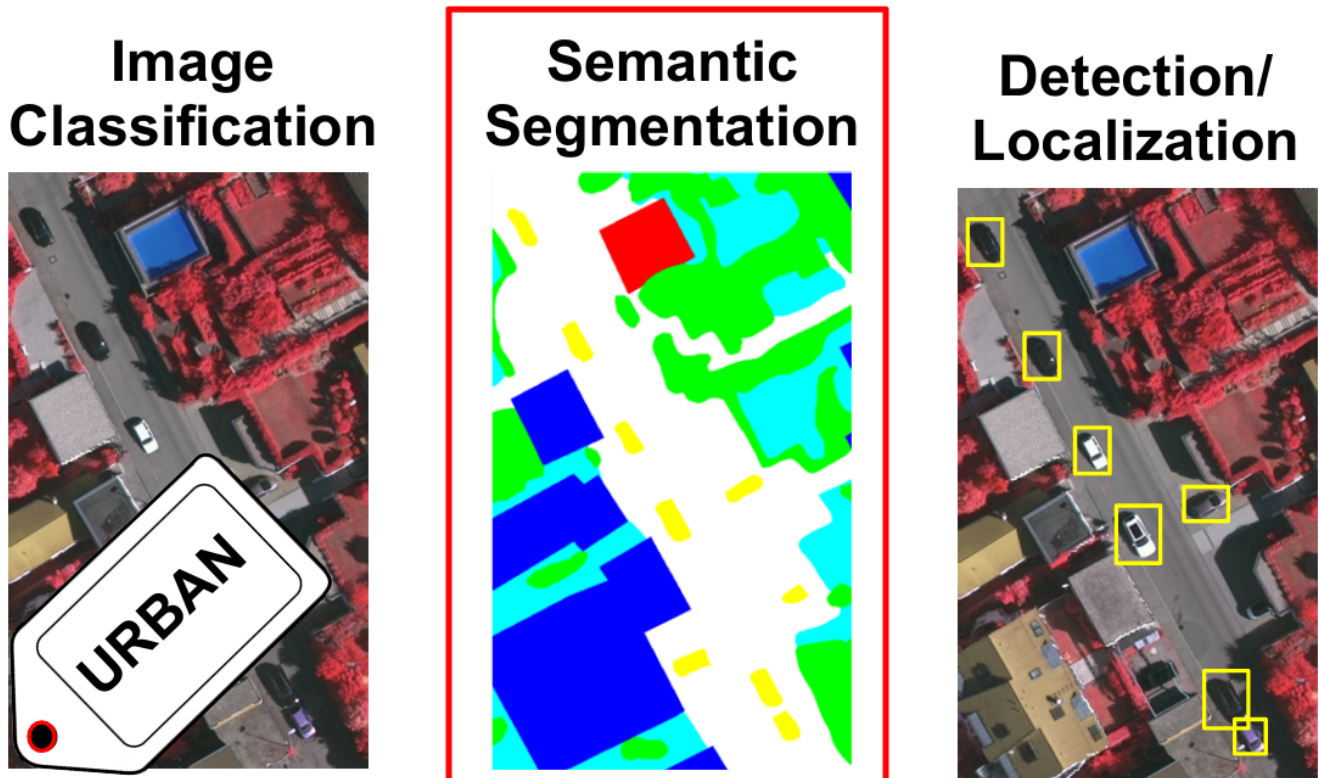


Figura 1: Áreas de visão computacional

Dentro do contexto de segmentação, vale diferenciar entre segmentação semântica e segmentação de instâncias. A segmentação semântica atribui uma classe a cada pixel, enquanto a segmentação de instâncias não apenas classifica os pixels, mas também distingue entre diferentes objetos da mesma classe.

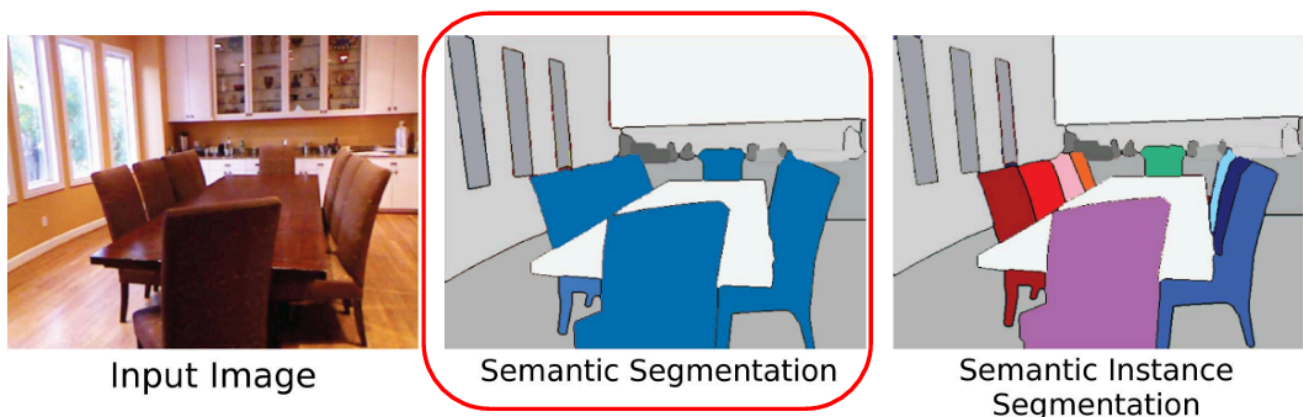


Figura 2: Diferença entre segmentação semântica e segmentação de instâncias

1.2 Métricas

Podemos utilizar algumas métricas para avaliar a performance de um modelo de segmentação semântica. As métricas mais comuns incluem:

Definição 1.2.1 (Intersection over Union (IoU)):

$$\text{IoU} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}} \quad (1)$$

Definição 1.2.2 (Precision):

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2)$$

Definição 1.2.3 (Average Precision): Dado que na minha imagem eu tenho mapeado K classes, a métrica de Average Precision (AP) é definida como a média das precisões de cada classe:

$$\text{AP} = \left(\frac{1}{K} \right) * \sum_{k=1}^K \text{Precision}_k \quad (3)$$

1.3 Abordagens progressivas

Vamos relembrar como é estruturada uma rede convolucional padrão para classificação de uma imagem.

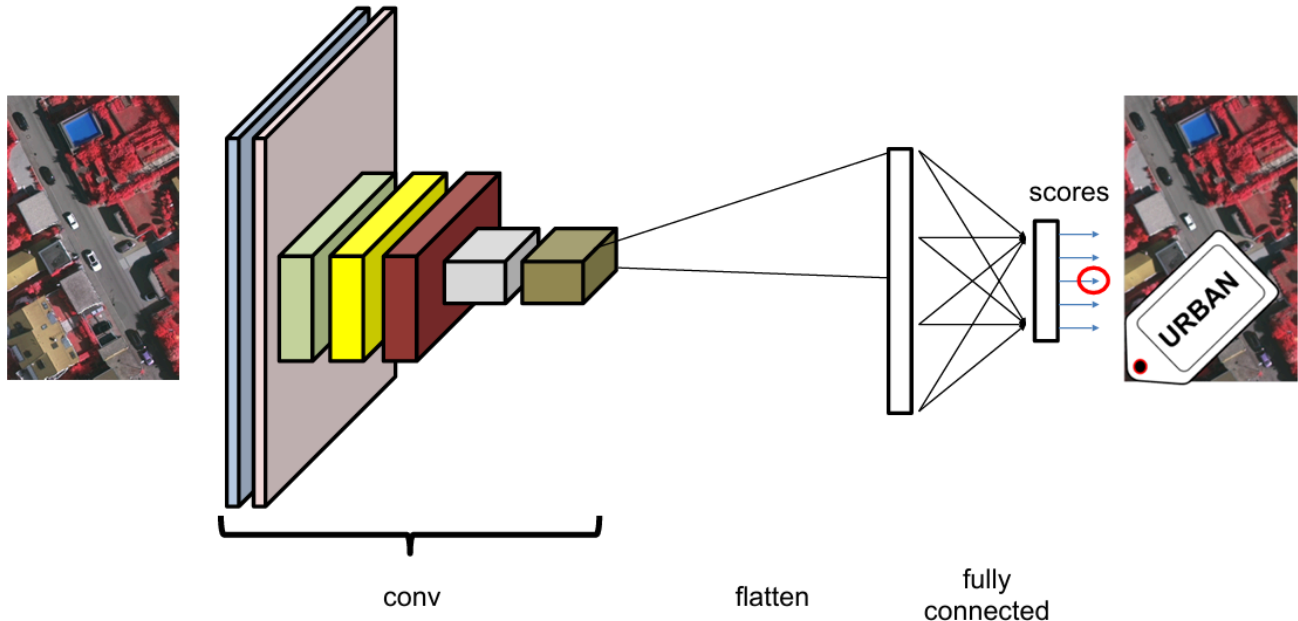


Figura 3: Estrutura de uma rede convolucional padrão

A imagem passa por uma série de canais de convolução, pooling e normalização, que extraem características relevantes. No final, temos uma camada totalmente conectada que produz a classificação final. Isso nos faz pensar em uma ideia, que tal termos uma janela que percorre a imagem e classifica cada pixel individualmente? Essa abordagem é conhecida como “sliding window” e é uma das primeiras tentativas de segmentação semântica.

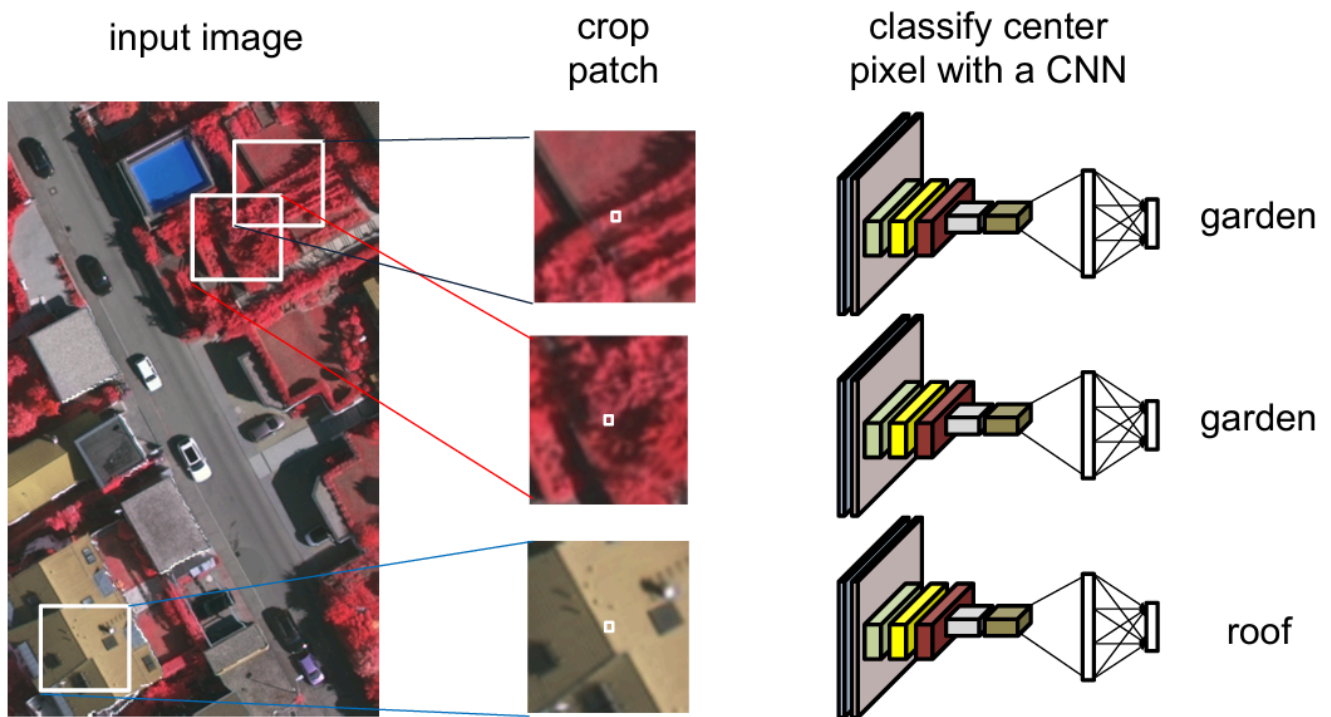


Figura 4: Abordagem de sliding window

No entanto, essa abordagem é computacionalmente cara e não aproveita o contexto global da imagem. Para superar essas limitações, surgiram as Fully Convolutional Networks (FCNs), que substituem as camadas totalmente conectadas por camadas convolucionais, permitindo que a rede produza mapas de segmentação diretamente.



Figura 5: Estrutura de uma Fully Convolutional Network (FCN)

Essa abordagem é interessante, já que permite que a rede aprenda a segmentar a imagem de forma mais eficiente, utilizando o contexto global e local. Além disso, as FCNs podem ser treinadas de forma end-to-end, o que simplifica o processo de treinamento. No entanto, as FCNs são MUITO pesadas, e por isso surgiu a ideia do **downsampling** e **upsampling** dentro da rede, onde basicamente diminuimos os tamanhos das features internas e depois vamos reconstruindo até o tamanho original da imagem. Essa abordagem é conhecida como **encoder-decoder**.

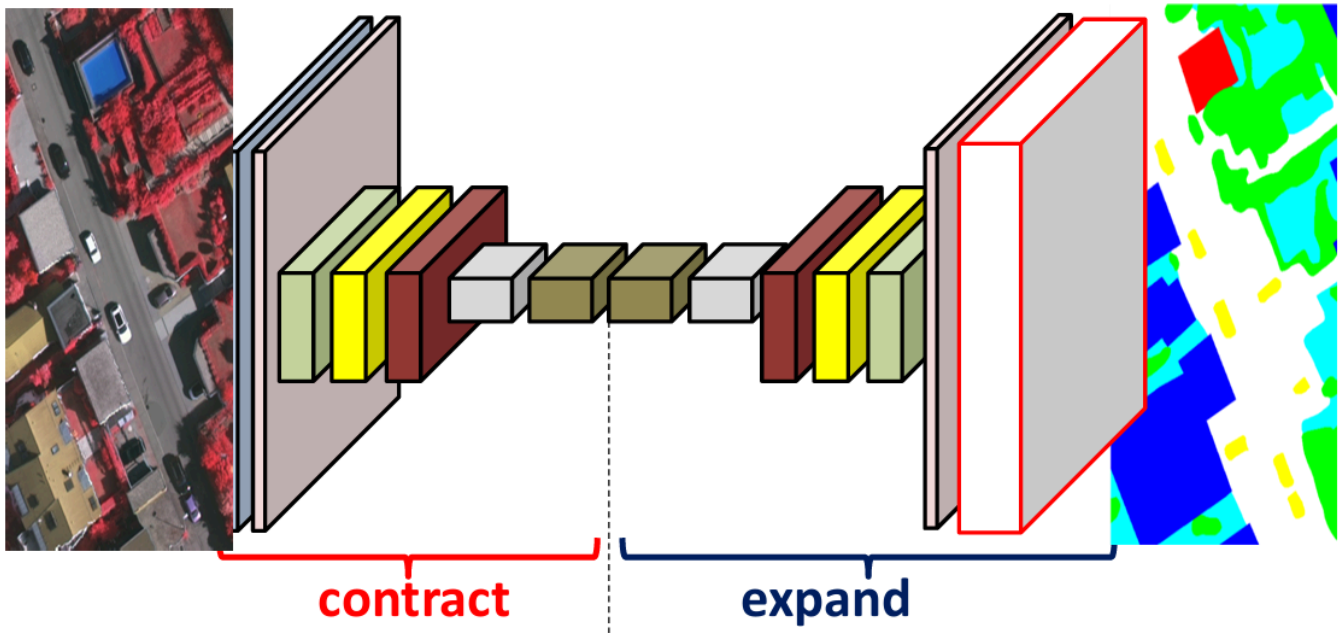


Figura 6: Estrutura de uma rede encoder-decoder

1.4 Upsampling

Antes de realmente mostrar as arquiteturas de redes encoder-decoder, vamos definir **bem** algumas técnicas de upsampling, assim não precisaremos interromper o raciocínio no meio do caminho

1.4.1 Max Unpooling

Relembrando o que é o **pooling**, ele é uma operação que reduz a dimensionalidade das features internas da rede, geralmente utilizando operações como **max pooling** ou **average pooling**. O **unpooling** é o processo inverso, onde tentamos reconstruir a dimensão original das features a partir das features reduzidas. No entanto, o **unpooling** não é uma operação trivial, pois não temos informações suficientes para reconstruir a dimensão original de forma precisa.

O que acontece é que, **antes** de fazer o **pooling**, nós guardamos os índices dos valores máximos (no caso do **max pooling**), e depois utilizamos esses índices para reconstruir a dimensão original durante o **unpooling**. Essa abordagem é conhecida como **max unpooling**.

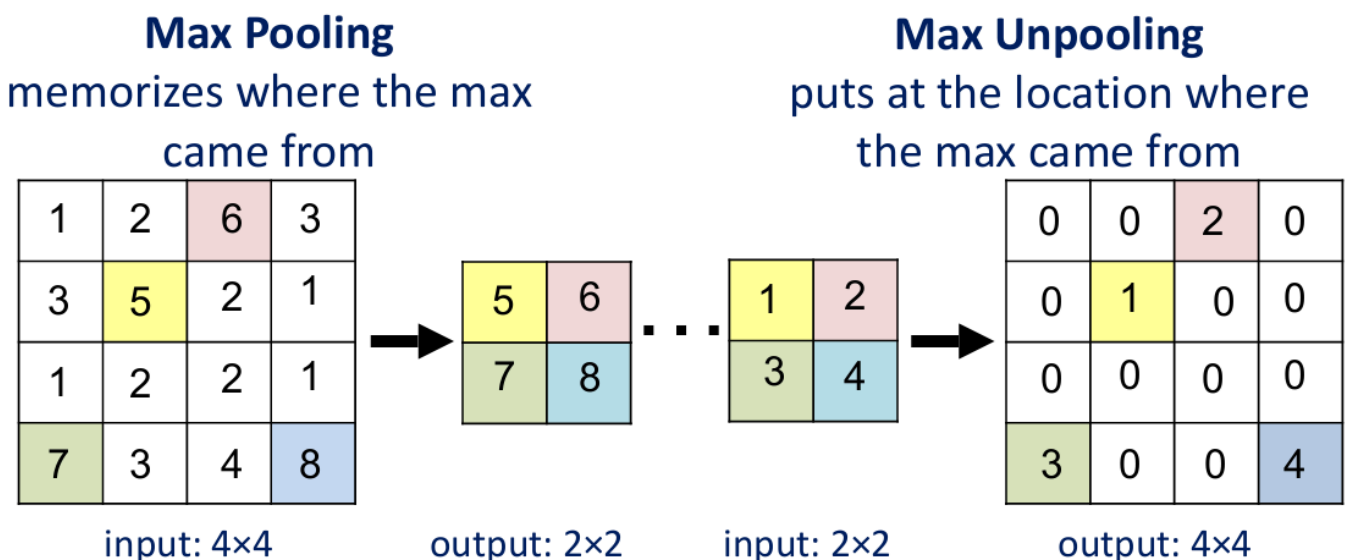


Figura 7: Exemplo de max unpooling

1.4.2 Transpose Convolution

A desvantagem do **max unpooling** é que ele depende dos índices dos valores máximos, o que pode limitar a capacidade da rede de aprender representações mais complexas. Uma alternativa é utilizar a **transpose convolution**, também conhecida como **deconvolution**. Essa operação é semelhante à convolução, mas ao invés de reduzir a dimensionalidade das features, ela aumenta.

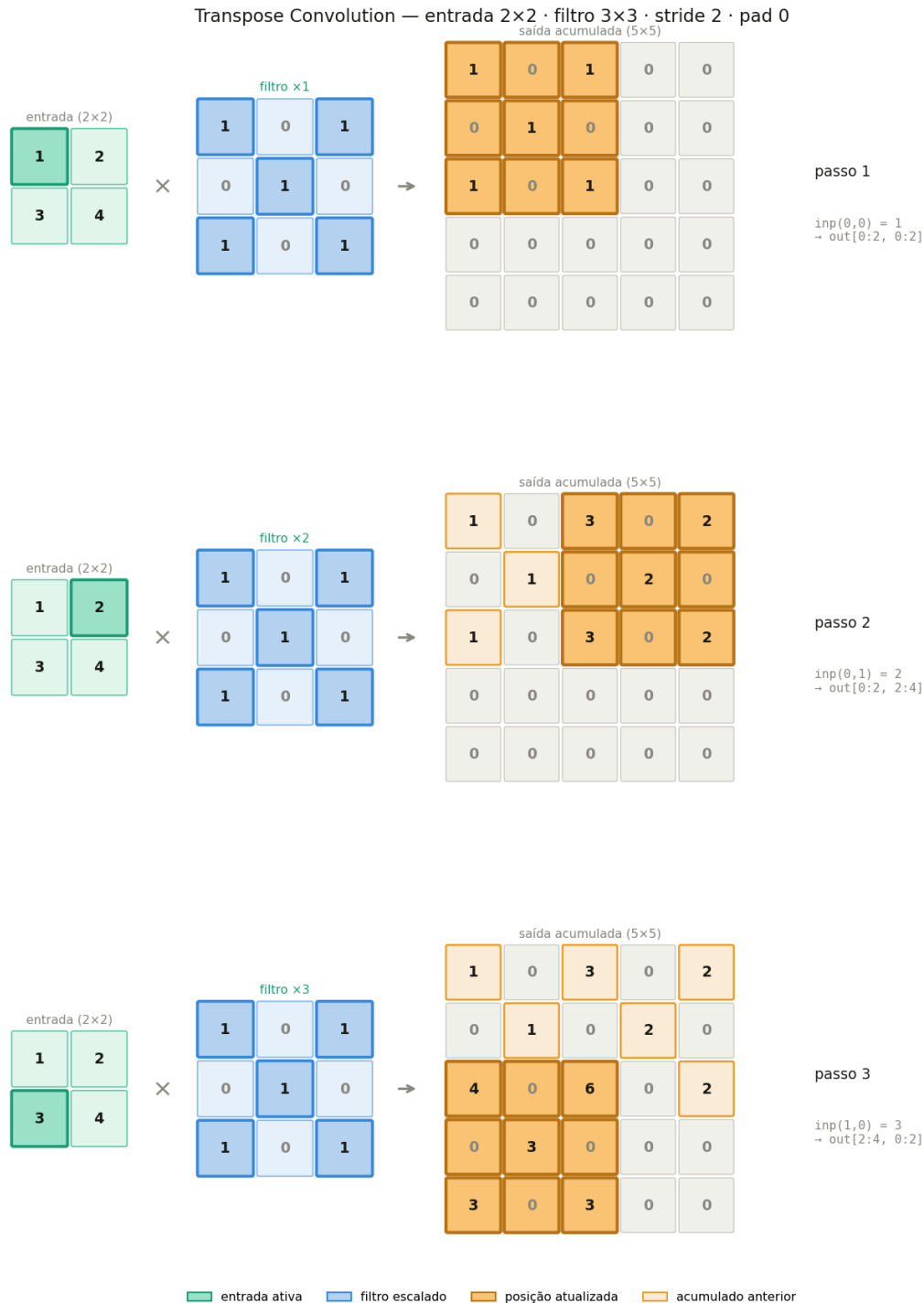


Figura 8: Exemplo de transpose convolution

Como vimos na disciplina de **machine learning**, podemos obter uma matriz de filtro chamada K_{col} a partir da operação *im2col*, que transforma a imagem em uma matriz de colunas. A **transpose convolution** é basicamente a operação inversa, onde aplicamos a matriz de filtro K_{col} na matriz obtida anteriormente

Se X é a entrada da convolução e K é o filtro tal que

$$Y = X * K \quad (4)$$

Já sabemos que

$$Y = K_{\text{col}} X_{\text{col}} \quad (5)$$

temos então que a transpose convolution é definida como

$$X_{\text{col}} = K_{\text{col}}^T Y \quad (6)$$

de tal forma que os X_{col} são reconstruídos a partir dos Y e do filtro K_{col} (não de forma perfeita pois há perda de informação na compressão do X para o Y , mas o objetivo é que a rede aprenda a reconstruir o X da melhor forma possível)

1.5 Contexto Global e Atrous Convolution

Antes de irmos de fato para as arquiteturas, temos que definir um conceito usado em algumas delas, se não vamos interromper o raciocínio no meio do caminho. O conceito é o de **contexto global**, que basicamente é a ideia de que, para classificar um pixel, precisamos levar em consideração não apenas os pixels vizinhos, mas também os pixels mais distantes da imagem. Só que usar filtros maiores consome mais memória, tempo de processamento e não permite que a rede aprenda a extrair características mais complexas. Para resolver esse problema, surgiu a ideia de **atrous convolution**, que é uma técnica que permite aumentar o tamanho do filtro sem aumentar o número de parâmetros da rede. A ideia é inserir “buracos” (ou **holes**) entre os pixels do filtro, permitindo que ele “veja” mais pixels da imagem sem aumentar o número de parâmetros.

Definição 1.5.1 (Atrous Convolution): Dado um filtro K de tamanho $k \times k$, e uma feature map X , a **atrous convolution** é definida como

$$Y(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X(i + r * m, j + r * n) K(m, n) \quad (7)$$

onde r é o **rate** de atrous convolution, que determina o espaçamento entre os pixels do filtro.

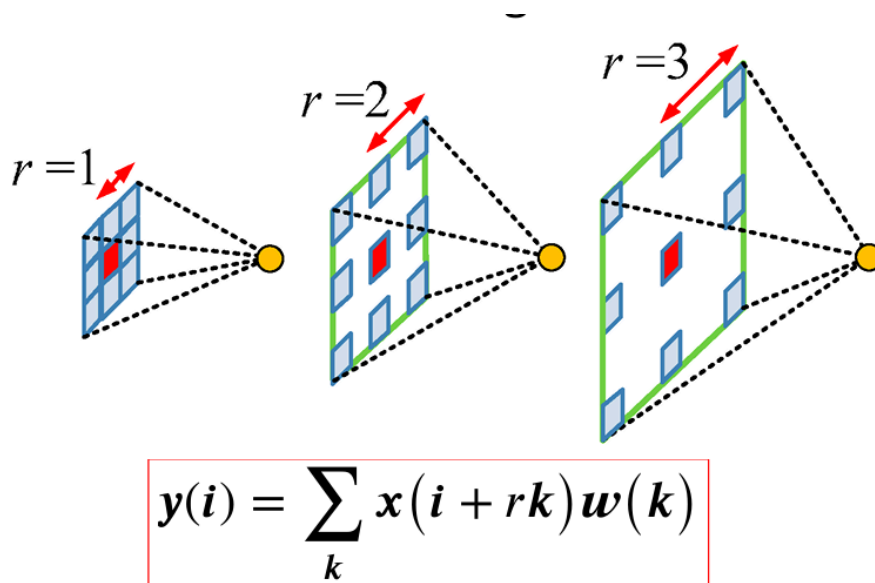


Figura 9: Exemplo de atrous convolution

1.6 Blocos Residuais

Normalmente, redes neurais são straight-to-the-point, nós temos a entrada x e a partir disso a rede modela uma função complexa F tal que

$$y = F(x) \quad (8)$$

no entanto, pode existir casos em que y é MUITO parecido com x com leves ajustes, e isso, surpreendentemente, pode dificultar muito o aprendizado da rede. Para consertar isso, os chamados **blocos residuais** foram introduzidos, de forma que a rede não aprende a relação direta entre x e y , mas sim a **diferença** entre eles (o quão diferente y é de x), ou seja, a rede aprende uma função F tal que

$$y = F(x) + x \quad (9)$$

O principal motivo dessa abordagem é o **gradiente no backpropagation**. Em redes comuns de deep-learning, o gradiente pode se tornar muito pequeno (ou até mesmo zero) à medida que é propagado para trás, dificultando o aprendizado. Com os blocos residuais, o gradiente pode fluir diretamente através da conexão de atalho, permitindo que a rede aprenda mais facilmente.

1.7 Arquiteturas

1.7.1 SegNet

A SegNet é uma arquitetura de rede neural convolucional projetada para segmentação semântica. Ela segue a arquitetura padrão que já demonstramos utilizando do método de max unpooling para realizar upscaling

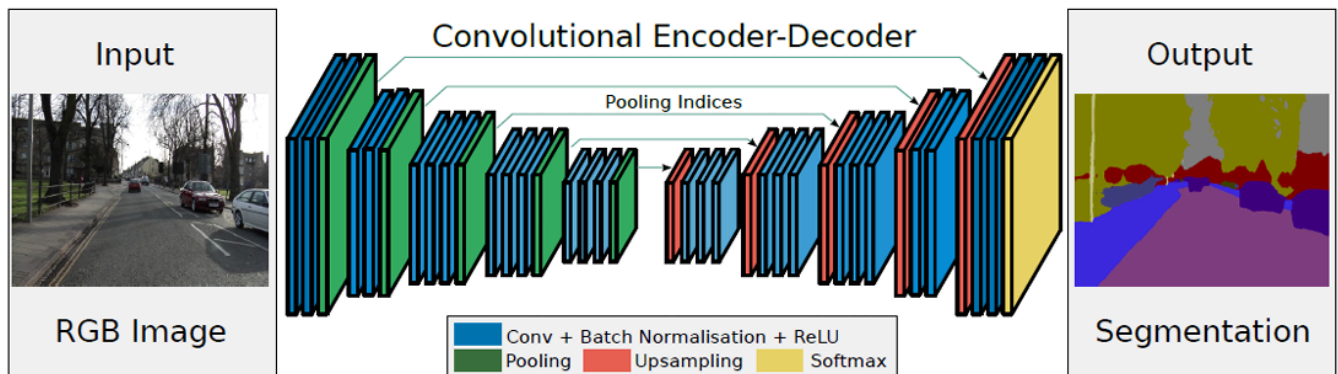


Figura 10: Arquitetura da SegNet

1.7.2 U-Net

Já na U-Net, a arquitetura é um pouco diferente, ela utiliza **skip connections** para conectar as camadas de downsampling com as camadas de upsampling, permitindo que a rede utilize informações de diferentes níveis de abstração para melhorar a segmentação.

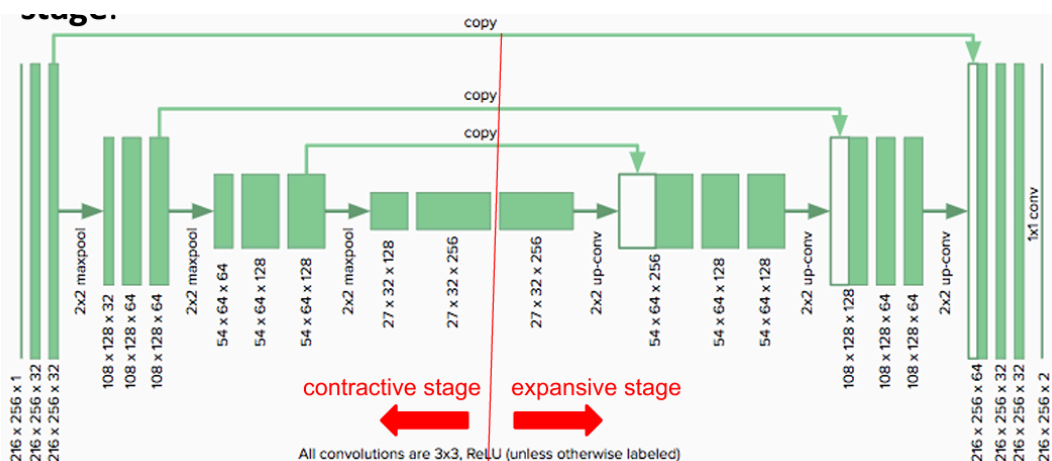


Figura 11: Arquitetura da U-Net

Nas camadas de upsampling, a U-Net utiliza **transpose convolution** para aumentar a dimensionalidade das features unida com um **aumento** nos canais das features. Após o transpose convolution, a U-Net concatena as features da camada correspondente de downsampling, permitindo que a rede utilize informações de diferentes níveis de abstração para melhorar a segmentação, como se ela falasse: “depois de reconstruir a imagem, eu obtive o seguinte mapa de feature, mas lá atrás antes de eu ter feito o downsampling, eu tinha obtido o seguinte mapa de feature, então vou juntar os dois para melhorar a segmentação” (por exemplo, se eu tenho uma imagem 32x32 na escala de cinza, com apenas um canal de cor, na hora do último upsampling, a camada logo após a transpose convolution terá 2 canais “de cor”, que seria o mapa obtido pela rede anteriormente e o mapa obtido na camada de upsampling).

1.7.3 ResUNet

Na ResUNet, a arquitetura é uma combinação da U-Net com blocos residuais, permitindo que a rede aprenda a diferença entre as features de downsampling e upsampling, melhorando ainda mais a segmentação.

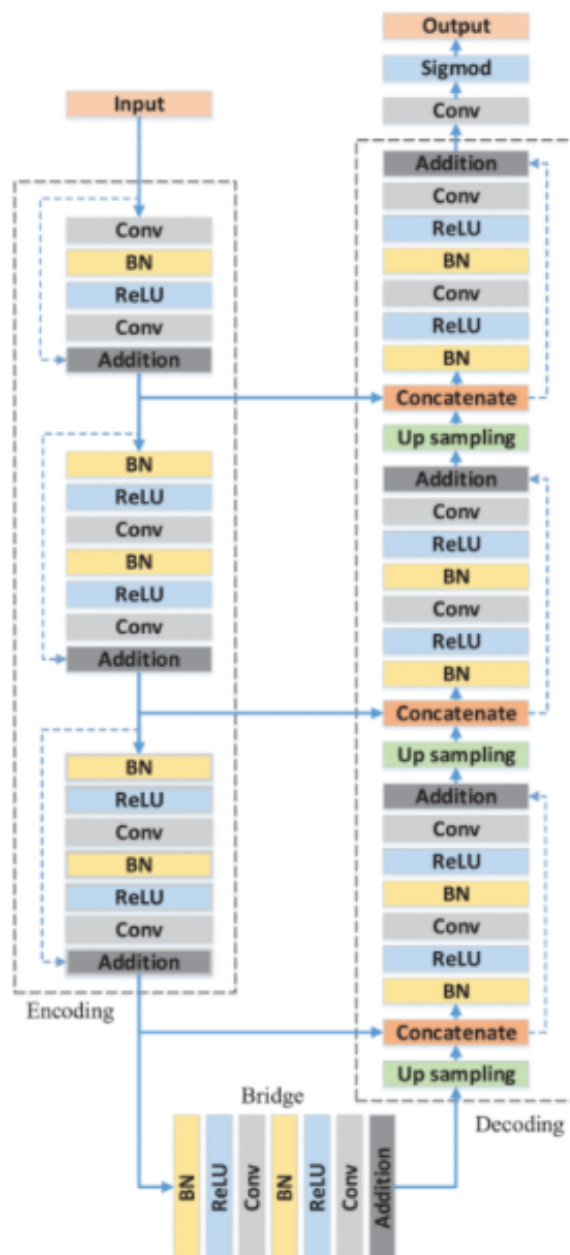


Figura 12: Arquitetura da ResUNet

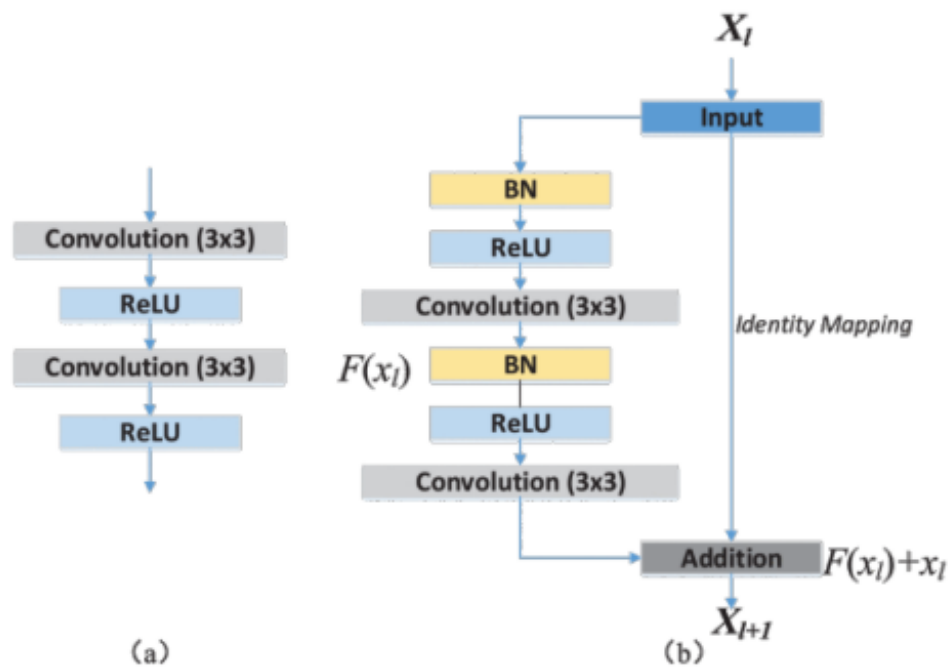


Figura 13: (a) Bloco padrão da UNet. (b) Bloco residual da ResUNet